While deducing a number type may be easy, how about the type of the `release_year` field? This field value is compared and matched to one of the two default date formats: `yyyy/MM/dd` or `yyyy-MM-dd`. If it matches, it is assigned a `date` data type.

Date format during dynamic mapping

Elasticsearch can deduce that a field is a `date` type if the values in the JSON document are provided in the format `yyyy-MM-dd` or `yyyy/MM/dd`, although the latter isn't an ISO date format. However, this flexibility is available only for dynamic mapping cases.

If we declare a field as a `date` type explicitly (using explicit mapping—more on this later), then unless we provide a custom format, the field adheres by default to the `strict_date_optional_time` format. This format conforms to an ISO date format: `yyyy-MM-dd` or `yyyy-MM-ddTHH:mm:ss`.

This guesswork is adequate for most cases but falls short if the data is slightly off the default track. Consider the `release_year` field in our movie document. If our document has a value for `release_year` in a different format (say, `ddMMyyyy` or `dd/MM/yyyy`), the dynamic mapping rule breaks down. Elasticsearch considers that value a `text` field rather than a `date` field.

Similarly, let's try indexing another document with the `rating` field set to 4 (we intend to provide ratings as decimal numbers). Elasticsearch determines that this field is a `long` data type as the value fits its "long" data type rule. Although we want the `rating` field to host floating-point data, Elasticsearch fell short in determining the appropriate type. Looking at the value `(4)`, it assumed that the field's data type is `long`.

In both cases, we have a schema with incorrect data types. Having incorrect types can cause problems in an application, making fields ineligible for sorting, filtering, and aggregating data. But this is the best that Elasticsearch can do when we use its dynamic mapping feature.

Bring your data—make your schema

While Elasticsearch is clever enough to derive mapping information based on our documents, so we don't have to worry about schemas, there is a chance that things will go badly and we will end up with an incorrect schema definition. Having a bad schema is asking for trouble.

Because we generally understand our domains well and know our data models inside and out, my advice is not to let Elasticsearch create schemas, especially in production environments. Instead, it's best to create our schemas up front, usually by creating a mapping strategy across the organization using mapping templates (we work through some mapping templates in chapter 6). Of course, if your use case includes data without a standard format, dynamic mapping can be a big help.